Actividad 7: Mini Juego "Piedra, Papel o Tijera" con Persistencia de Datos (struct + fstream)

Proyecto PIEDRAPAPELOTIJERA

Ian Quiroga, Santino Laudati y Dante Iglesias

EEST N.º 1 “Eduardo Ader” - Vicente López

Laboratorio de Programación - 5.º Año 3.º A-B

Profesor: Mansilla Muñoz York Elías

Yamil De Jesus Ganduglia

2026

Fecha de entrega: 4 de noviembre de 2026

Repositorio de GitHub: https://github.com/pachusUwu/piedrapapelotijera.git

Introducción

Un error frecuente de los programadores que recién empiezan es confundir la memoria RAM, que es volátil y pierde los datos al cerrar el programa, con el almacenamiento secundario (disco rígido o de estado sólido), que conserva los archivos de forma permanente. En esta actividad desarrollamos un mini juego de Piedra, Papel o Tijera contra la "inteligencia artificial" de la consola, en el que cada partida se registra en una estructura `Partida` y se guarda inmediatamente en el archivo de texto `data/historial_partidas.txt`.

Gracias a la biblioteca `<fstream>`, el programa escribe en el disco con `ofstream` en modo *append* (`ios::app`) y recupera el historial completo con `ifstream`, incluso después de cerrar y volver a abrir la consola. El informe describe el diseño, las mejoras que agregamos, la demostración de persistencia y las respuestas al cuestionario técnico de control.

Desarrollo

Integrantes y tareas realizadas

Tabla 1

Distribución de tareas del grupo

Integrante	Tareas realizadas
Ian Quiroga	Programación de `src/main.cpp`, validación de entradas, continuidad del ID entre sesiones, ruta `data/` para el archivo persistente, compilación con `-std=c++11` y pruebas.
Santino Laudati	Redacción del informe APA v7, cuestionario técnico, manual del usuario y reflexión sobre la aplicación al P.I.A.
Dante Iglesias	Demostración de persistencia con capturas, manual del programador, CHANGELOG, `.gitattributes`, README y sincronización Dual-Remote (GitHub + GitLab).

Diseño del programa (R2 – núcleo)

El programa se organiza en un menú de tres opciones y cuatro funciones modulares. La estructura agrupa los cinco datos de una partida:

```
struct Partida {
    int idPartida;
    string jugador;
    string eleccionJugador;
    string eleccionIA;
    string resultado;
};
```

Las dos líneas clave son la apertura del archivo para escritura en modo *append* y la apertura para lectura:

```
ofstream archivo(RUTA_HISTORIAL, ios::app);   // escribir al final, sin borrar
ifstream archivo(RUTA_HISTORIAL);             // leer el historial guardado
```

Cada partida se guarda como una línea en formato CSV (`1,Ian Quiroga,Piedra,Tijera,Victoria`) y la lectura separa los campos con `getline(archivo, campo, ',')`.

Mejoras agregadas al código base

- **Ruta `data/historial_partidas.txt`:** el código base guardaba en `src/`; se movió a `data/` para respetar la estructura del repositorio.
- **Continuidad del ID:** al iniciar, `contarPartidasEnDisco()` cuenta las líneas existentes, de modo que al reabrir el juego la numeración continúa (#4, #5…) en

lugar de repetir el #1.

- **Validación de jugada:** en el código base cualquier número distinto de 1 o 2 se tomaba como Tijera; ahora se rechazan los valores fuera de 1–3.
- **Entradas no numéricas:** si se escriben letras en el menú, `cin` queda en estado de error y el menú se repetía infinitamente; se agregó `limpiarEntrada()` con `cin.clear()` y `cin.ignore()`.

Demostración de persistencia

Se compiló con `g++ -std=c++11 src/main.cpp -o src/piedrapapelotijera.exe`. En una primera sesión se jugaron tres partidas (una por cada integrante) y se salió con la opción 3. Luego se cerró la ventana de la consola, se abrió una consola nueva, se ejecutó otra vez el programa y se eligió la opción 2: el historial apareció completo, leído desde el disco.

Figura 1

Primera partida jugada: Ian Quiroga eligió Piedra contra Tijera de la IA y la partida se guardó en disco

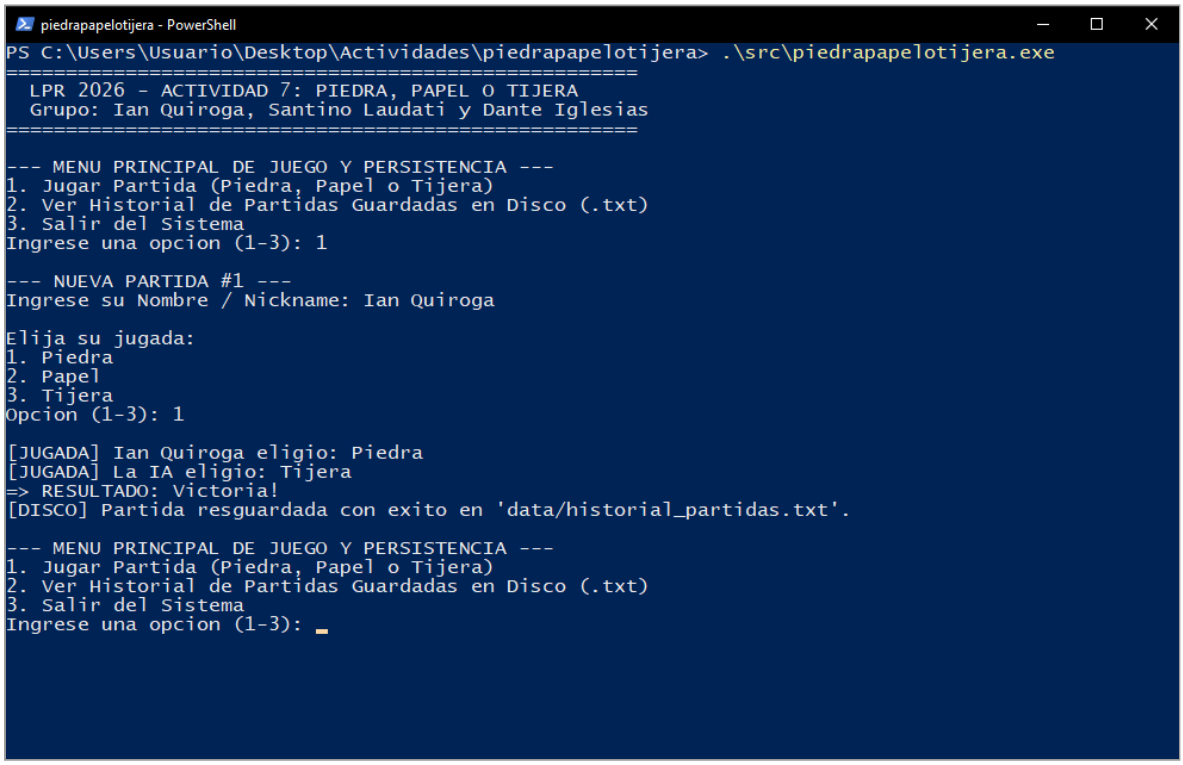


Figura 2

Fin de la primera sesión: tercera partida, validación de una opción inválida y salida del sistema

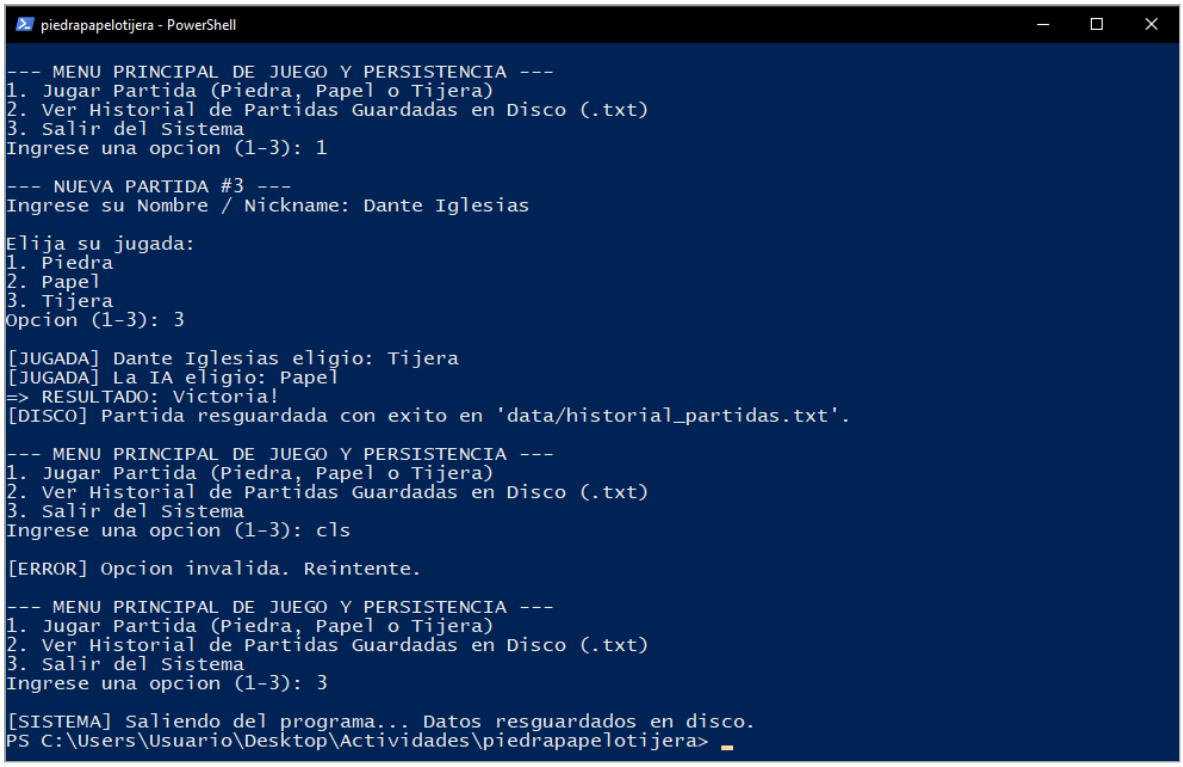


Figura 3

Consola reabierta: la opción 2 recupera desde el disco las tres partidas de la sesión anterior

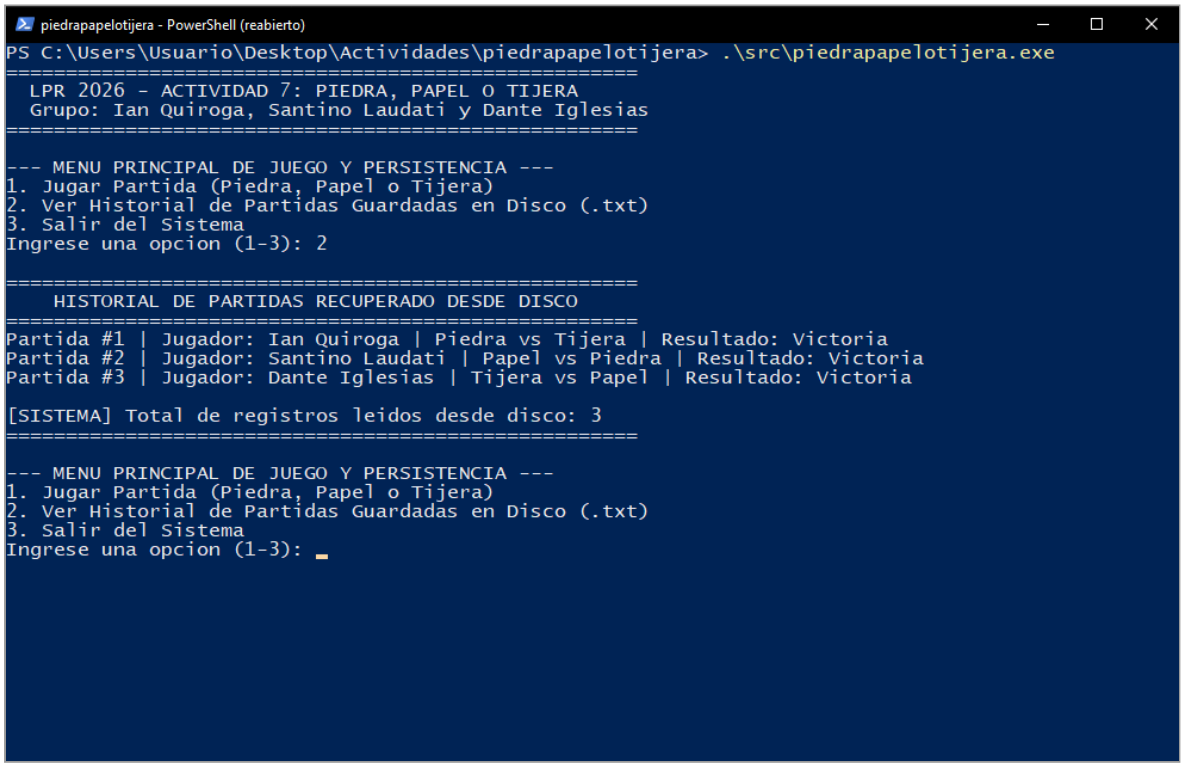
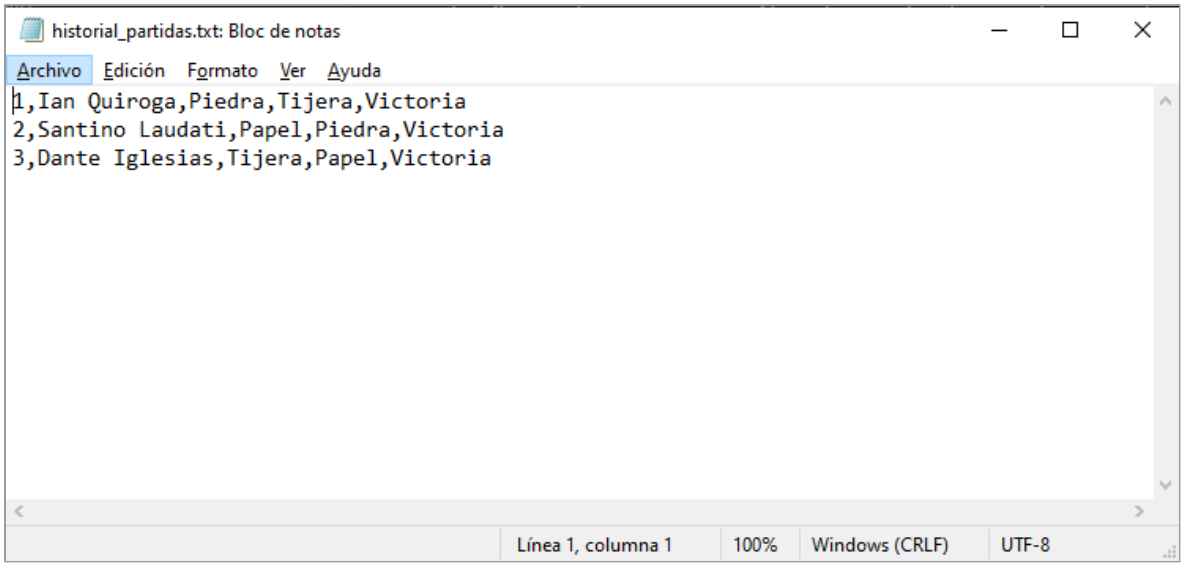


Figura 4

Archivo data/historial_partidas.txt abierto en el Bloc de notas, en formato CSV



Cuestionario técnico de control

1. RAM vs. Disco. Porque los datos de las partidas no quedan solamente en variables, que viven en la RAM y se borran al terminar el programa, sino que la función `guardarEnDisco()` los escribe en el archivo `historial_partidas.txt`, que está en el almacenamiento secundario (no volátil). Al volver a abrir el ejecutable, la opción 2 abre ese archivo con `ifstream` y lee las líneas guardadas, por lo que el historial sigue disponible aunque la consola se haya cerrado.

2. Uso de `ios::app`. Sin `ios::app`, `ofstream` abre el archivo en modo de salida normal, que lo **trunca**: borra todo su contenido antes de escribir. Cada partida nueva reemplazaría a la anterior y el archivo terminaría conteniendo una sola línea, la de la última partida jugada. Con `ios::app` cada escritura se agrega al final y se conserva el historial completo.

3. Manejo del búfer (`cin.ignore()`). Al elegir la opción del menú con `cin >> opcion`, se lee el número pero el Enter (`\n`) queda pendiente en el búfer de entrada. `getline()` lee hasta encontrar un `\n`; si no se descarta antes, lo encuentra de inmediato, devuelve un nombre vacío y la pregunta del nombre se "saltea". `cin.ignore()` elimina ese carácter para que `getline()` espere lo que escribe el jugador.

4. Agrupación con `struct`. La estructura `Partida` mantiene juntos todos los datos de una misma partida, por lo que las funciones reciben un solo parámetro (`const Partida&`

`p`) en lugar de cinco variables sueltas. Esto reduce errores (por ejemplo, pasar los argumentos en otro orden), hace el código más legible, permite agregar un campo nuevo sin cambiar la firma de todas las funciones y, al pasarse por referencia constante, evita copias innecesarias en la RAM.

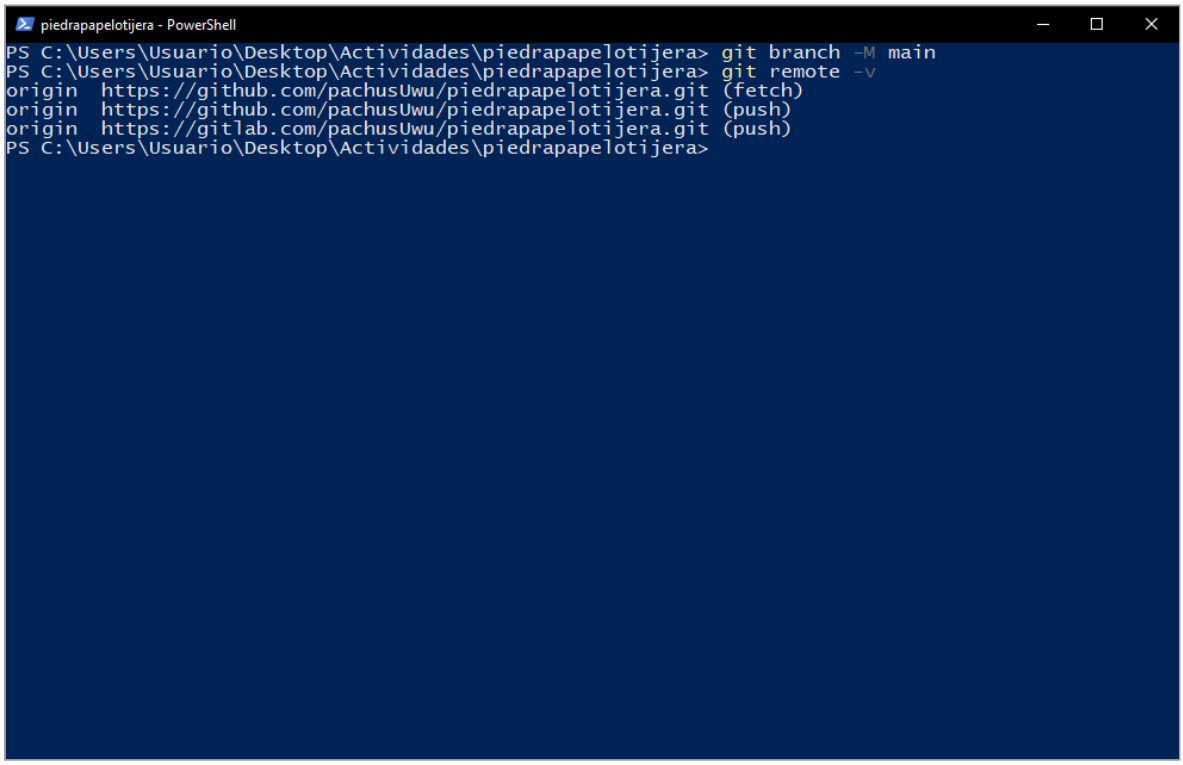
5. Aplicación al P.I.A. En el prototipo del Proyecto Integrador guardaríamos un registro (*log*) de eventos del dispositivo: lecturas del sensor de temperatura con su número de lectura, el valor medido y si superó el umbral de alarma, por ejemplo `15,DHT11,27.5,Normal`. Con el mismo formato CSV el archivo puede abrirse luego en una planilla de cálculo para hacer gráficos de las mediciones durante la Expotécnica.

Repositorio y sincronización Dual-Remote

El proyecto se versionó con Git (ramas `main` y `development`) y se configuró el remoto `origin` con dos direcciones de envío, una en GitHub y otra en GitLab, de modo que `git push` sube el código a ambos servidores. Las direcciones públicas figuran en las Referencias.

Figura 5

Configuración Dual-Remote verificada con git remote -v



Conclusión

La actividad dejó muy clara la diferencia entre la memoria RAM y el disco: mientras las variables desaparecen al cerrar el programa, el archivo de texto conserva el historial de partidas entre sesiones. Comprobamos en la práctica que el modo `ios::app` es indispensable para no perder los registros anteriores y que `cin.ignore()` resuelve el problema del Enter que queda en el búfer.

Además, al probar el juego con datos incorrectos encontramos y corregimos comportamientos del código base (IDs repetidos al reabrir el juego, jugadas inválidas tomadas como Tijera y un menú que se repetía sin fin al ingresar letras). Esta técnica de persistencia en archivos CSV es directamente aplicable al registro de lecturas de sensores de nuestro Proyecto Integrador.

Referencias

cppreference.com. (s.f.). *std::ios_base::openmode*. https://en.cppreference.com/w/cpp/io/ios_base/openmode

GeeksforGeeks. (2025). *Structures in C++*. https://www.geeksforgeeks.org/cpp/structures-in-cpp/

GeeksforGeeks. (2025). *How to use getline() in C++ when there are blank lines in input?* https://www.geeksforgeeks.org/cpp/how-to-use-getline-in-c-when-there-are-blank-lines-in-input/

Git. (s.f.). *git-remote documentation*. https://git-scm.com/docs/git-remote

Mansilla Muñoz, Y. E. (2026). *Actividad 7: Mini juego "Piedra, papel o tijera" con persistencia de datos (struct + fstream)* [Material de clase]. E.E.S.T. N.° 1 "Eduardo Ader".

Microsoft. (2024). *Programación de iostream* [Flujos de archivo]. Microsoft Learn. https://learn.microsoft.com/es-es/cpp/standard-library/iostream-programming

Quiroga, I., Laudati, S., & Iglesias, D. (2026a). *piedrapapelotijera* [Repositorio de código fuente]. GitHub. https://github.com/pachusUwu/piedrapapelotijera

Quiroga, I., Laudati, S., & Iglesias, D. (2026b). *piedrapapelotijera* [Repositorio espejo]. GitLab. https://gitlab.com/pachusUwu/piedrapapelotijera